Z3 Linear Arithmetic Quick Reference: LRA / LIA (Python)

Start every program with `from z3 import *`. **LIA** (linear integer arithmetic) works over the integers $\mathbb{Z}$; **LRA** (linear real arithmetic) works over the reals $\mathbb{R}$, and Z3 computes with **exact fractions**, never floating point. "Linear" means variables are only added and multiplied by constants; multiplying two variables (`x*y`) is *nonlinear* arithmetic, which is harder (see section 8).

1. Declaring variables and constants

Operation	Syntax	Example	Meaning
Integer variable	`Int('x')`	`x = Int('x')`	unknown integer
Several integers	`Ints('x y z')`	`x, y, z = Ints('x y z')`	declare many at once
Real variable	`Real('a')`	`a = Real('a')`	unknown real number
Several reals	`Reals('a b')`	`a, b = Reals('a b')`	declare many at once
Integer constant	`IntVal(n)`	`IntVal(5)`	the integer 5 (plain `5` also works next to a Z3 term)
Exact fraction	`Q(p, q)` or `RealVal("p/q")`	`Q(1, 3)`	exactly 1/3
Real constant	`RealVal(v)`	`RealVal(2.5)`	the real 5/2

2. Arithmetic

Operation	Syntax	Example	Meaning
Add / subtract	`x + y`, `x - y`	`x + y == 10`	exact, no overflow
Multiply by a constant	`c * x`	`3*x + 2*y <= 7`	linear term
Negate	`-x`	`-x >= 0`	
Sum of a list	`Sum(list)`	`Sum([x, y, 1])`	x + y + 1
Absolute value	`Abs(x)`	`Abs(x - y) <= 2`	|x − y|
Real division	`a / b` (Real)	`RealVal(7) / 2`	exact: 7/2
Integer division	`x / y` (Int)	`IntVal(7) / 2`	integer quotient: 3 (see section 9)
Integer remainder	`x % y`	`x % 2 == 0`	x is even; result is always ≥ 0
Int → Real	`ToReal(x)`	`ToReal(x) / 2`	5 becomes 5/2 instead of 2
Real → Int	`ToInt(a)`	`ToInt(a)`	floor: 3.7 → 3, −3.7 → −4
Is it an integer?	`IsInt(a)`	`IsInt(a)`	true iff the real a is a whole

number

3. Comparisons

Operation	Syntax	Example	Meaning
Equal / not equal	`x == y`, `x != y`	`x == 2*y`	builds a constraint, not a Python True/False
Less / greater	`< <= > >=`	`0 <= x`	usual order on $\mathbb{Z}$ or $\mathbb{R}$
All different	`Distinct(...)`	`Distinct(x, y, z)`	pairwise different

4. Combining constraints

Operation	Syntax	Example
And / Or / Not	`And(c1, c2)`, `Or(c1, c2)`, `Not(c)`	`Or(x <= 0, x >= 10)`
Implication	`Implies(c1, c2)`	`Implies(x > 0, y > 0)`
If-then-else	`If(c, t, e)`	`If(x > y, x, y)` (maximum)
Quantifiers	`ForAll([v], c)`, `Exists([v], c)`	`ForAll([a], Implies(a > 0, 2*a > a))`

Do **not** use Python's `and`, `or`, `not` on Z3 expressions — use `And`, `Or`, `Not`.

5. Solving and reading results

```
from z3 import *

x, y = Ints('x y')
s = Solver()
s.add(x + y == 10, x - y >= 2, y >= 0)   # add constraints

if s.check() == sat:                        # sat / unsat / unknown
    m = s.model()
    print(m)                                # e.g. [y = 0, x = 10]
    print(m[x].as_long())                   # Int value as a Python int
    print(m.eval(2*x + y))                  # evaluate any expression
else:
    print("no solution")
```

Operation	Syntax
Add constraints / check	`s.add(c1, c2, ...)`, `s.check()` → `sat`, `unsat`, `unknown`
Int value from a model	`m[x].as_long()`
Real value from a model	`m[a].as_fraction()` (exact, e.g. 1/3) or `m[a].as_decimal(5)` (e.g. 0.33333?)
Evaluate / simplify	`m.eval(expr)`, `simplify(expr)`
Prove a property	add its **negation**; `unsat` means the property always holds

6. Optimization

```
o = Optimize()
o.add(x + y <= 10, x >= 0, y >= 0, x - y >= 2)
h = o.maximize(x + 2*y)          # or o.minimize(...)
print(o.check(), o.model(), h.value())   # sat [y = 4, x = 6] 14
```

7. Same formula, different answer: LIA vs LRA

Constraint	Over Int (LIA)	Over Real (LRA)
`3*x == 1`	**unsat** (no integer is 1/3)	**sat**, x = 1/3
`And(x > 0, x < 1)`	**unsat**	**sat**, e.g. x = 1/2
`x / 2` with x = 5	2	5/2

LRA is decided by the **simplex** method; LIA additionally needs branch-and-bound and cutting planes, so it can be slower.

8. Nonlinear arithmetic

Constraint	Result
`x*x == 2` over Int	unsat
`a*a == 2` over Real	sat, a = −1.4142135623? (an exact algebraic number, printed approximately; `?` marks the approximation)

Nonlinear real arithmetic is decidable; nonlinear integer arithmetic is **undecidable**, so Z3 may answer `unknown` or run for a very long time.

9. Common mistakes

- **Integer division rounds, and not always toward zero.** For Int, `x / y` and `x % y` satisfy x = y·q + r with 0 ≤ r < |y|: so −7 / 2 = −4 and −7 % 2 = 1 (Python's `//` agrees here, C would give −3 and −1). Use `ToReal(x) / 2` if you want 5/2.
- **Python fractions are floats.** `RealVal(1/3)` gives 3333333333333333/10000000000000000, not 1/3. Write `Q(1, 3)` or `RealVal("1/3")`.
- **Mixing Int and Real** silently converts the Int with `ToReal`: `x + a` becomes `ToReal(x) + a`. Declare every variable with the sort you mean.
- **Division by zero is not an error.** `a / 0` is some unspecified value, so `a / 0 == 5` is sat. Add `b != 0` yourself when dividing by a variable.
- `x == y` is a Z3 constraint, not a Python Boolean — do not use it in a Python `if`; use `s.check()` or `simplify`.
- **Real values in a model** are exact fractions; `as_long()` only works for Int values — use `as_fraction()` or `as_decimal(k)` for Reals.

10. Optimization in more detail

Operation	Syntax	Meaning
Create / constrain	`o = Optimize()`, `o.add(c1, c2, ...)`	used like a `Solver`
Set an objective	`h = o.minimize(e)`, `h = o.maximize(e)`	returns a handle; nothing is solved yet
Solve	`o.check()`	must be called before reading anything; `sat` / `unsat` / `unknown`
Read the optimum	`h.value()`, `o.model().eval(e)`	the objective value, or the objective expression evaluated in the optimal model
Bounds so far	`h.lower()`, `h.upper()`	interval known to contain the optimum
Temporary constraints	`o.push()`, `o.pop()`	add constraints, solve, then undo them (also on `Solver`)
Time limit	`o.set("timeout", ms)`	gives up after *ms* milliseconds; `check()` then returns `unknown` (also on `Solver`)

Details worth knowing: an unbounded objective gives `oo` (or `-oo`) as its value. With a strict bound such as `x > 0`, minimizing `x` gives the value `epsilon`, an infinitesimal; the strict bound is never attained. Several objectives are optimized in the order you add them (lexicographic). The optimum is exact, as with any Z3 real.

11. Exact numbers in Python

Operation	Syntax	Meaning
Python exact fraction	`from fractions import Fraction` `Fraction(1, 4)`, `Fraction("0.25")`	exact rational that supports `+ - * /`, comparisons, `abs`, `max`, `min`
Model value → Fraction	`m[a].as_fraction()`, `o.model().eval(e).as_fraction()`	the exact value of a real as a `Fraction`
Fraction → Z3 term	`RealVal(str(f))`, `Q(f.numerator, f.denominator)`	puts a Python `Fraction` into a constraint
Fraction → display	`float(f)`, `round(f, 3)`, `m[a].as_decimal(3)`	approximate output for printing only; do not feed it back into a constraint
Variable missing from model	`m.eval(e, model_completion=True)`	a variable the solver never needed is left out of the model; this gives it a default value instead of returning the variable itself

`Fraction(0.1)` is not 1/10, because `0.1` is already a float before `Fraction` sees it. Pass a string or two integers.

12. Building constraints from data

Operation	Syntax	Meaning
Generated variable names	`Real(f"z_{j}")`, `[Real(f"x{i}") for i in range(n)]`	one fresh variable per index; the name is what makes it distinct, so reusing a name gives the *same* variable
List of constraints	`s.add(*cons)`, `And(cons)`, `Or(cons)`	`*` unpacks a Python list into separate arguments; `And` / `Or` also accept a list
Max / min of terms	`If(a > b, a, b)`	Python's `max` and `min` do not work on Z3 terms; they only work on plain numbers and `Fraction`s
Binary switch	`p = Int('p')`, `Or(p == 0, p == 1)`	an integer restricted to two values
Relaxed switch	`p = Real('p')`, `And(0 <= p, p <= 1)`	the same variable allowed to take any value in between
Bool → number	`If(b, 1, 0)`	a Z3 Boolean is not a number

Absolute value as a constraint	`Abs(e) >= d` or `Or(e >= d, e <= -d)`	the two forms say the same thing

An `Int` times a real constant, or added to a `Real`, is converted automatically (see section 9), so `u * p` with an integer `p` is fine inside a real constraint.

13. Running and timing

Operation	Syntax	Meaning
Command-line flags	`argparse.ArgumentParser()`, `add_argument("--A", action="store_true")`	the flag is `False` unless given on the command line; read it as `args.A`
Wall-clock time	`import time`; `t = time.perf_counter()`	call it before and after a `check()` and subtract
Reproducible randomness	`import random`; `random.seed(0)`, `random.randint(a, b)`	a fixed seed gives the same numbers on every run
Plain-Python check	write the same function on ordinary numbers	compare its output with `m.eval(...)` at a point to test your Z3 encoding